

System Architecture & Data Flow

How the seven Forkly services talk to each other — from customer login, through menu, cart, order, payment and kitchen, to live delivery tracking.

Microservices

REST + JWT

.NET 8 · Vue 3 · PostgreSQL

Team reference v1

The team & their services

User Management

Izzuwan — login, JWT, roles, profile, addresses

Menu Management

Amirul — menu items: name, description, price, picture

Order Service

Hanif — create order, SST + total, history, reorder

Payment

Aiman — charge reference + total, confirm payment

Kitchen

Zul — crew queue, mark preparing / done / shipped

Notification & Tracker

Alia — status feed + mock delivery timer

Golden rule: every service owns its **own database**. Nobody reads another team's tables directly — they call its API. That isolation is why each part can be built and deployed independently.

01 API (REST) vs gRPC

The single question that decides which one to use: **who is calling whom?**

USE FOR: BROWSER → BACKEND

REST (HTTP + JSON)

- Human-readable JSON
- Browsers call it directly (`fetch/axios`)
- Test in Swagger / Postman
- Everyone already knows it

USE FOR: BACKEND → BACKEND

gRPC (HTTP/2 + protobuf)

- Compact binary, very fast
- Browsers **cannot** call it directly (needs a proxy)
- Strict shared `.proto` contract
- Great for high-volume service-to-service calls

Recommendation for our team — start 100% REST. It unblocks everyone, is browser-friendly, and is trivial to test. The repo already has `.proto` files scaffolded, so gRPC stays available as a Phase-2 optimisation — but it should never block you from shipping.

Hard rule: any call coming **from the Vue frontend must be REST** — a browser can't speak raw gRPC.

02 System map — who owns what

VUE FRONTEND (browser)

login · landing · order · profile · admin · kitchen-crew · payment · tracker

▼ all browser → backend calls = REST + JWT ▼

User

Izzuwan

- login / JWT
- roles
- profile
- addresses

Menu

Amirul

- menu items
- name / desc
- price / picture
- admin CRUD

Order

Hanif

- create order
- SST + total
- order history
- reorder

Payment

Aiman

- take payment
- ref + amount
- confirm

Kitchen

Zul

- crew queue
- mark prep
- done / ship

Tracker

Alia

- status feed
- mock timer
- Preparing → Delivered

Order — menu lookup —> **Menu** REST or gRPC

Order — charge: reference + total —> **Payment** REST or gRPC

Order — new kitchen ticket —> **Kitchen** REST or gRPC

Payment — paid! start cooking —> **Kitchen** REST or gRPC

Kitchen / Payment — status update —> **Order (source of truth)** REST or gRPC

Order / Kitchen — status events —> **Tracker** REST or gRPC

03 End-to-end data flow (happy path)

LOGIN

- 1 Customer opens the **login page** (Izzuwan).
- 2 POST `/api/auth/login {email, password}` → REST.
- 3 User service returns a **JWT** (contains `userId` + **role**) and the user profile JSON.
- 4 Frontend routes by role → Customer: landing · Admin: dashboard · Crew: kitchen screen.

BROWSE & CART

- 1 Landing + order page load the menu: GET `/api/menu` → name, description, unitPrice, imageUrl (Amirul).
- 2 Customer adds items to the cart (cart lives in the browser / Pinia store).

PLACE ORDER

- 1 Order page → POST `/api/orders {items:[...]}` → **your Order Service**.
- 2 Order Service computes **Subtotal, SST (6%), Total**, generates a **Reference** (FRK-1001), saves order + items (status = Pending).
- 3 Order Service hands off → **Payment** (reference + total) and → **Kitchen** (ticket: items + user).

PAY

- 1 Frontend opens the **Payment page** (Aiman) showing reference + total.
- 2 Customer pays → Payment confirms.
- 3 Payment notifies → **Order** (status = Paid), → **Kitchen** (start cooking), → **Tracker** (order is live).

KITCHEN

- 1 Crew logs in → sees the ticket on Zul's screen.
- 2 Taps **Start** → status = Preparing → notifies Order + Tracker.
- 3 Taps **Completed** → status = Completed → notifies Order + Tracker.
- 4 Taps **Out for delivery** (rider picked up) → status = OutForDelivery → notifies Tracker.

TRACK & HISTORY

- 1 Tracker (Alia) shows live status; on OutForDelivery it starts a **mock timer** (random 15–30 min or preset), counts down, then shows **Delivered**.
- 2 Customer profile (Izzuwan) shows history via GET `/api/orders/user/{userId}` → your Order Service.

04 Order status lifecycle

Five services read/write this one field, so the words must match exactly. The **Order Service is the single source of truth**; everyone else updates status through its API.



STATUS	SET BY	MEANS
Pending	Order (Hanif)	Order created, awaiting payment
Paid	Payment (Aiman)	Payment confirmed
Preparing	Kitchen (Zul)	Crew started cooking
Completed	Kitchen (Zul)	Food ready
OutForDelivery	Kitchen (Zul)	Rider picked up → tracker timer starts
Delivered	Tracker (Alia)	Mock timer elapsed
Cancelled	Order / Payment	Allowed from Pending / Paid

05 Data contracts — what each person hands off

Agree on these JSON shapes and everyone can build in parallel.

Amirul (Menu) → everyone · a menu item

```
// Order page shows name/desc/price/picture. Order STORES only a snapshot of id+name+price
{ "menuId":1, "name":"Beef Burger", "description":" ... ",
  "unitPrice":10.00, "imageUrl":"https:// ... /burger.jpg", "available":true }
```

Hanif (Order) → Aiman · payment request

```
// total ALREADY includes the 6% SST – Aiman just charges it
{ "reference":"FRK-1001", "orderId":12, "userId":7,
  "totalAmount":53.00, "currency":"MYR" }
```

Hanif (Order) → Zul · kitchen ticket

```
{ "reference":"FRK-1001", "orderId":12, "userId":7,
  "items":[ {"itemName":"Beef Burger","quantity":2},
    {"itemName":"Coffee","quantity":1} ] }
```

Aiman (Payment) → Order + Kitchen · payment confirmed

```
{ "reference":"FRK-1001", "orderId":12, "status":"Paid", "paidAt":" ... " }
```

Zul (Kitchen) → Order + Tracker · status update

```
{ "orderId":12, "status":"Preparing" } // then "Completed", then "OutForDelivery"
```

Alia (Tracker) → UI · mock tracker

```
{ "orderId":12, "status":"OutForDelivery", "etaMinutes":22, "deliveredAt":null }
// etaMinutes is the MOCK – random or preset, no real GPS
```

06 Roles & login routing

The User service puts a **role** inside the JWT. The frontend reads it and routes; every backend already trusts the same token.

ROLE	LANDS ON	CAN SEE
Customer	Landing page	Menu, cart, order, payment, tracker, own profile + history
Admin	Admin dashboard	User mgmt, Menu mgmt, Reports, Kitchen overview (read-only)
Crew NEW — Izzuwan adds	Kitchen crew screen (Zul)	Order queue + status buttons only

07 Recommended build order

Sequenced by dependency, so nobody is blocked waiting on a contract that doesn't exist yet.

- 1 **Izzuwan — User:** login + JWT + roles. Everyone needs the token first. *(mostly done)*
- 2 **Amirul — Menu:** menu API. The landing/order pages need it.
- 3 **Hanif — Order:** built ☒ — wire it to Amirul's menu; add **Reference**, **OutForDelivery/Delivered** statuses, and a status-update endpoint.
- 4 **Aiman — Payment:** needs the order reference + total.
- 5 **Zul — Kitchen:** needs orders (Hanif) + paid signal (Aiman) + the Crew role (Izzuwan).
- 6 **Alia — Tracker:** last in the chain; consumes status events from everyone.

Note for the profile page: orders live in the Order Service's own database, not in Izzuwan's tables. So the profile page should **fetch** history from `GET /api/orders/user/{userId}` (pull model), not read its own DB.